Python Continue Statement

Last Updated : 13 Mar, 2026

The continue statement in Python is a loop control statement that skips the rest of the code inside the loop for the current iteration and moves to the next iteration immediately.

When executed, the code following continue in the loop is not executed for that iteration. **Example:**

```
for i in range(1, 11):
    if i == 6:
        continue
    print(i, end=" ")
```

```
1 2 3 4 5 7 8 9 10
```

Output

```
1 2 3 4 5 7 8 9 10
```

Explanation: When i == 6, the continue statement executes, skipping the print operation for 6.

Syntax

```
while True:
    ...
    if x == 10:
        continue
    print(x)
```

Examples

Example 1: Skipping specific characters in a string

```
for char in "GeeksforGeeks":
    if char == "e":
        continue
    print(char, end=" ")
```

```
G k s f o r G k s
```

Output

```
G k s f o r G k s
```

Explanation: Whenever char == 'e', the continue statement executes, skipping the print function for that iteration.

Example 2. Using continue in nested loops

```
a = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]

for row in a:
    for num in row:
        if num == 3:
            continue
        print(num, end=" ")
```

```
1 2 4 5 6 7 8 9
```

Output

```
1 2 4 5 6 7 8 9
```

Explanation: continue statement skips printing 3 and moves to the next iteration, so all numbers except 3 are printed in a single line.

Example 3. Using continue with a while loop

```
i = 0
while i < 10:
    if i == 5:
        i += 1  # ensure the loop variable is incremented to avoid infinite loop
        continue
    print(i)
    i += 1
```

```
0
1
2
3
4
6
7
8
9
```

Output

```
0
1
2
3
4
6
7
```

```
8
9
```

Explanation: When i == 5, the continue statement skips printing and jumps to the next iteration.

When to Use the Continue

The continue statement is useful when you need to skip specific iterations while still executing the rest of the loop:

- **Skipping Specific Values:** Ignore certain values without breaking the loop.
- **Filtering Data Dynamically:** Exclude elements that do not meet a condition.
- **Optimizing Loop Performance:** Avoid unnecessary computations for some iterations.

For more loop control statements, refer to:

- Python break Statement
- Python pass Statement

Continue In Python

Visit Course

Comment

N nikhila... 81

Explore

Python Fundamentals

Python Data Structures

Advanced Python

Data Science with Python

Web Development with Python

GeeksforGeeks
Sanchhaya Education Private Limited

Corporate & Communications Address:
A-143, 6th Floor, Sovereign Corporate Tower, Sector- 136, Noida, Uttar Pradesh (201305)

Registered Address:
K 061, Tower K, Gulshan Vivante Apartment, Sector 137, Noida, Gautam Buddh Nagar, Uttar Pradesh, 201305

GET IT ON Google Play
Download on the App Store

@GeeksforGeeks, Sanchhaya Education Private Limited, All rights reserved